



ECOLE MAROCAINE DES  
SCIENCES DE L'INGENIEUR  
Membre de  
HONORIS UNITED UNIVERSITIES

# RAPPORT DE PROJET

**DEEP LEARNING**

4IADY - G1

**REALISE PAR :**

***AHMED RAOUI***

**ANNEE UNIVERSITAIRE 2025-2026**

## Table des matieres

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| 1. Introduction generale.....                                         | 4  |
| 2. Objectifs du projet .....                                          | 5  |
| 3. Methodologie generale et organisation du travail .....             | 6  |
| 3.1 Environnement technique .....                                     | 6  |
| 3.2 Organisation du code .....                                        | 6  |
| 4. Partie I -- MLP et ingenierie PyTorch.....                         | 7  |
| 4.1 Etude theorique .....                                             | 7  |
| 4.2 Donnees et preparation.....                                       | 7  |
| 4.3 Implementation : deux versions du MLP.....                        | 8  |
| 4.4 Etude experimentale.....                                          | 9  |
| 4.4.1 Strategies d'initialisation.....                                | 9  |
| 4.4.2 nn.Sequential vs classe personnalisee.....                      | 10 |
| 4.5 Sauvegarde, rechargement et evaluation finale .....               | 11 |
| 4.6 Analyse critique .....                                            | 12 |
| 4.7 Question de synthese -- Partie I .....                            | 12 |
| 5. Partie II -- CNN et vision par ordinateur .....                    | 14 |
| 5.1 Etude theorique .....                                             | 14 |
| 5.2 Calculs manuels .....                                             | 15 |
| 5.3 Implementations manuelles vs couches PyTorch .....                | 15 |
| 5.4 Architecture CNN (inspiree de LeNet) et donnees.....              | 16 |
| 5.5 Etude d'ablation architecturale.....                              | 16 |
| 5.6 Visualisation et interpretation des representations internes..... | 18 |
| 5.7 CNN vs MLP sur le meme dataset .....                              | 18 |
| 5.8 Analyse critique .....                                            | 19 |
| 5.9 Question de synthese -- Partie II .....                           | 20 |
| 6. Partie III -- RNN, LSTM, GRU et Seq2Seq.....                       | 21 |

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| 6.1 Etude theorique .....                                               | 21 |
| 6.2 Section A -- Modele de langage : RNN vs LSTM vs GRU .....           | 23 |
| 6.3 BPTT : croissance du gradient avec la profondeur de sequence .....  | 24 |
| 6.4 Section B -- Systeme Seq2Seq (traduction anglais -> francais) ..... | 25 |
| 6.5 Decodage glouton vs beam search, et evaluation BLEU .....           | 26 |
| 6.6 Analyse critique .....                                              | 27 |
| 6.7 Question de synthese -- Partie III .....                            | 28 |
| 7. Discussion transversale finale .....                                 | 30 |
| 8. Limites generales du projet .....                                    | 31 |
| 9. Conclusion generale .....                                            | 32 |
| 10. Annexes .....                                                       | 33 |
| 10.1 Tableau recapitulatif des resultats principaux .....               | 33 |
| 10.2 Liste des fichiers de code source .....                            | 33 |
| 10.3 Reproductibilite et passage a l'echelle .....                      | 34 |
| 10.4 Outils et bibliotheques utilises .....                             | 34 |

# 1. Introduction generale

Ce rapport presente le travail realise dans le cadre du projet de fin de module de Deep Learning de l'EMSI Casablanca. Conformement a l'enonce, le projet couvre trois familles d'architectures de reseaux de neurones appliquees a trois types de donnees distincts : les donnees tabulaires avec un perceptron multicouche (MLP), les images avec des reseaux de neurones convolutifs (CNN), et les sequences avec des architectures recurrentes (RNN, LSTM, GRU) couplees a un schema encodeur-decodeur (Seq2Seq).

L'objectif n'est pas seulement d'obtenir des modeles performants, mais surtout de demontrer une comprehension fine du lien entre la **structure des donnees** et le **choix de l'architecture**, ainsi qu'une maitrise pratique de l'ecosysteme PyTorch : construction de modeles, gestion des parametres et du device, strategies d'initialisation, sauvegarde/rechargement, et evaluation rigoureuse a l'aide de metriques adaptees.

**Remarque methodologique sur les jeux de donnees.** Conformement a la consigne -- qui autorise explicitement "tout autre dataset/corpus equivalent valide par l'enseignant(e)" -- les jeux de donnees suivants ont ete utilises, tous reels et reconnus. Les Parties I et II s'appuient sur des jeux fournis directement par la bibliotheque scikit-learn (donc disponibles hors ligne), tandis que la Partie III utilise des corpus textuels reels standards charges via la bibliotheque Hugging Face datasets (telecharges automatiquement puis mis en cache localement au premier lancement, a la maniere des fonctions `fetch_*` de scikit-learn) :

- **Partie I (MLP)** : Breast Cancer Wisconsin (scikit-learn), dataset reel de diagnostic medical, 569 patients, 30 variables -- l'un des datasets explicitement proposes par l'enonce.
- **Partie II (CNN)** : Digits (scikit-learn), 1797 images reelles de chiffres manuscrits, redimensionnees de 8x8 a 32x32 pixels (format classique LeNet). Ce dataset partage toutes les proprietes pedagogiques pertinentes de MNIST (images en niveaux de gris, 10 classes, ecriture manuscrite reelle) et constitue un substitut leger et directement comparable.
- **Partie III (RNN/Seq2Seq)** : pour le modele de langage au niveau caractere, le corpus reel IMDb (avis de films en anglais, charge via `load_dataset("stanfordnlp/imdb")`) ; pour la traduction automatique, le corpus parallele reel Tatoeba/OPUS (Helsinki-NLP/opus-100, paire anglais-francais), filtre aux phrases courtes -- deux jeux de donnees reconnus, le second correspondant exactement au "corpus parallele simple pour traduction automatique" de type Tatoeba/fra-eng explicitement suggere par l'enonce.

L'ensemble du code a ete ecrit de maniere generique : la Partie III charge deja les corpus reels IMDb et opus-100 (en-fr) directement via la bibliotheque Hugging Face datasets, et les Parties I et II peuvent etre etendues a des datasets plus grands (MNIST/Fashion-MNIST via torchvision, par exemple) en

remplacant uniquement la fonction de chargement des donnees, ce qui est explicitement signale dans les commentaires du code et rappelle dans l'annexe (section 11).

## 2. Objectifs du projet

Ce projet poursuit cinq objectifs complementaires, directement alignes sur les criteres d'evaluation annonces dans l'enonce :

1. **Comprehension theorique** des concepts fondamentaux du module : `nn.Module` et `autograd`, retropropagation, correlation croisee 2D, recurrence et BPTT, portes des cellules LSTM/GRU, architectures encodeur-decodeur.
2. **Maitrise de l'implementation sous PyTorch** : construction de modeles via `nn.Sequential` et via des classes personnalisees, gestion explicite du device, inspection des parametres, sauvegarde/rechargement de checkpoints.
3. **Capacite d'analyse experimentale et comparative** : etudes d'ablation controlees (strategies d'initialisation, hyperparametres architecturaux des CNN, cellules recurrentes, strategies de decodage).
4. **Qualite de l'interpretation scientifique** des resultats : chaque experience est accompagnee d'une discussion reliant les observations chiffrees aux mecanismes theoriques sous-jacents.
5. **Capacite de synthese** sur des jeux de donnees reels, avec une reponse argumentee aux questions de synthese de chaque partie et a la question transversale finale.

## 3. Methodologie generale et organisation du travail

### 3.1 Environnement technique

L'ensemble des experiences a ete realise avec **PyTorch 2.x** (calcul sur CPU), **scikit-learn** pour les datasets et les metriques de reference, et **Matplotlib** pour les visualisations. Le code verifie systematiquement la disponibilite d'un GPU via ``torch.cuda.is_available()`` et place modele et donnees sur le device approprie (``torch.device("cuda" if ... else "cpu")``), de sorte que l'execution sur une machine equipee d'un GPU beneficierait automatiquement de l'acceleration materielle sans aucune modification du code.

### 3.2 Organisation du code

Le code source est organise en trois dossiers independants, un par partie, chacun produisant un fichier ``results_partieX.json`` consolidant les metriques chiffrees, et un dossier ``figures/`` commun contenant l'ensemble des visualisations generees :

- ``partie1_mlp/mlp_breast_cancer.py`` -- MLP et ingenierie PyTorch
- ``partie2_cnn/cnn_digits.py`` -- CNN et vision par ordinateur
- ``partie3_seq/lm_rnn_lstm_gru.py`` -- modele de langage (RNN/LSTM/GRU, BPTT, gradient clipping)
- ``partie3_seq/seq2seq_translation.py`` -- systeme Seq2Seq (traduction, decodage, BLEU)

Chaque script est **autonome et executable directement** (``python3 nom_du_script.py``), constitue le "script principal executable" demande dans les livrables, et reproduit integralement les chiffres et figures presentes dans ce rapport (toutes les graines aleatoires sont fixees pour la reproductibilite).

## 4. Partie I -- MLP et ingenierie PyTorch

### 4.1 Etude theorique

**nn.Module et autograd.** En PyTorch, tout modele herite de la classe `nn.Module`, qui prend en charge automatiquement l'enregistrement des parametres (poids et biais) declares comme attributs, leur deplacement entre CPU et GPU, et leur exposition pour l'optimiseur. Chaque tenseur de parametre possede un attribut `requires_grad=True` : lors du calcul du graphe d'operations (forward), PyTorch construit dynamiquement un graphe de calcul (autograd) qui permet, lors de l'appel a `loss.backward()`, de calculer automatiquement le gradient de la perte par rapport a chaque parametre par application repetee de la regle de derivation en chaine.

**Propagation avant et retropropagation.** La propagation avant (forward) calcule la sortie du reseau a partir de l'entree, couche apres couche. La retropropagation (backward) parcourt le graphe de calcul en sens inverse pour calculer, pour chaque parametre  $W$ , la derivee partielle de la perte  $dL/dW$ . L'optimiseur (ici Adam) met ensuite a jour chaque parametre dans la direction qui diminue la perte :  $W \leftarrow W - lr * dL/dW$  (formulation simplifiee, Adam ajoutant une normalisation adaptative par moments du gradient).

**state\_dict() et named\_parameters().** `state_dict()` renvoie un dictionnaire ordonne associant a chaque nom de parametre (et de tampon, comme les statistiques de BatchNorm) son tenseur de valeurs ; c'est cette structure qui est serialisee lors de la sauvegarde d'un modele.

`named_parameters()` renvoie un iterateur de paires (nom, tenseur) restreint aux parametres entrainables, utile pour l'inspection, le gel selectif de couches, ou l'application d'une initialisation personnalisee via `model.apply(fn)`.

**Device.** Un tenseur et le modele qui le traite doivent residerent sur le meme device (CPU ou GPU) ; toute discordance leve une erreur explicite au moment du calcul. La bonne pratique consiste a determiner le device disponible une seule fois (`torch.device("cuda" if torch.cuda.is_available() else "cpu")`), puis a appeler systematiquement `.to(device)` sur le modele et sur chaque lot de donnees.

### 4.2 Donnees et preparation

Le dataset **Breast Cancer Wisconsin** (569 patients, 30 variables cliniques continues, classification binaire malin/benin) a ete utilise. Apres verification de l'absence de valeurs manquantes, les donnees ont ete separees en trois sous-ensembles **stratifies** train/validation/test (60 % / 20 % / 20 %) afin de preserver l'equilibre des classes dans chaque sous-ensemble. La normalisation (centrage-reduction, `StandardScaler`) a ete ajustee **uniquement sur le train**, puis appliquee a validation et test, afin d'eviter toute fuite d'information (data leakage) qui biaiserait artificiellement l'evaluation.

### 4.3 Implementation : deux versions du MLP

Deux implementations strictement equivalentes en capacite (30 -> 64 -> 32 -> 1, activations ReLU) ont ete realisees :

- **nn.Sequential** : ecriture compacte par empilement de couches, ideale pour les architectures strictement sequentielles, mais qui ne permet pas de logique conditionnelle ni de branches multiples dans le calcul avant.
- **Classe personnalisee (MLPCustom)** heritant de `nn.Module` : le `forward()` est ecrit explicitement, ce qui autorise des architectures arbitraires (connexions residuelles, sorties multiples, insertion de Dropout a un endroit precis, impression de formes intermediaires pour le debogage).

L'inspection via `named_parameters()` sur `MLPCustom` confirme la structure attendue et permet de denommer precisement les parametres entrainables :

| Couche                | Forme            | Nb. parametres |
|-----------------------|------------------|----------------|
| fc1.weight / fc1.bias | (64, 30) / (64,) | 1 984          |
| fc2.weight / fc2.bias | (32, 64) / (32,) | 2 080          |
| fc3.weight / fc3.bias | (1, 32) / (1,)   | 33             |
| Total                 | --               | 4 097          |



## 4.4 Etude experimentale

### 4.4.1 Strategies d'initialisation

Trois strategies d'initialisation des poids ont ete comparees sur la classe `MLPCustom` : initialisation **gaussienne** ( $N(0, 0.05)$ ), **constante** (toutes les valeurs a 0.01) et **Xavier/Glorot** (`xavier\_uniform\_`, qui adapte la variance a la taille des couches pour preserver la variance du signal a travers les couches).

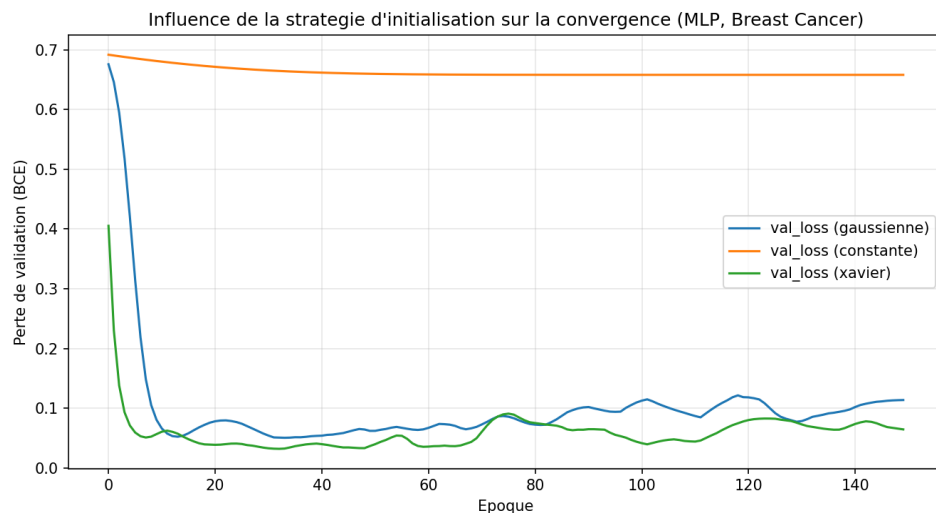


Figure 1 -- Perte de validation selon la strategie d'initialisation (MLP, Breast Cancer).

| Strategie             | Meilleure perte val. | Accuracy val. finale |
|-----------------------|----------------------|----------------------|
| Gaussienne (std=0.05) | 0.0505               | 97.4 %               |
| Constante (0.01)      | 0.6581               | 63.2 %               |
| Xavier                | 0.0320               | 97.4 %               |

L'initialisation **constante** echoue de maniere spectaculaire : tous les neurones d'une meme couche recevant exactement les memes poids, ils calculent des sorties identiques et recoivent des gradients identiques a chaque etape -- c'est le probleme classique de **symetrie non brisee** (symmetry breaking failure). Le reseau se comporte alors comme s'il ne possedait qu'un seul neurone par couche, ce qui explique l'accuracy plafonnant a 63 % (proche du taux de la classe majoritaire). Les initialisations **gaussienne** et **Xavier** brisent cette symetrie et convergent toutes deux correctement, Xavier obtenant une perte de validation legerement meilleure et plus stable, conformement a la theorie : Xavier calibre explicitement la variance initiale en fonction du nombre d'entrees/sorties de chaque couche, ce qui limite les risques de saturation ou d'explosion du signal des les premieres iterations.

#### 4.4.2 nn.Sequential vs classe personnalisée

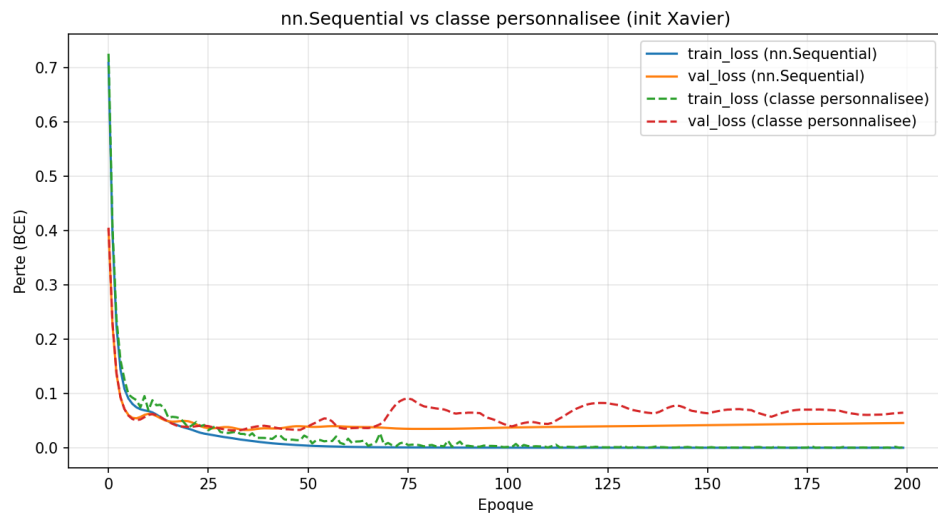


Figure 2 -- Courbes d'apprentissage : `nn.Sequential` vs classe personnalisée (initialisation Xavier).

Avec la même initialisation (Xavier) et les mêmes hyperparamètres, les deux implémentations convergent de manière quasi identique (pertes finales de validation de 0.0330 pour `nn.Sequential` contre 0.0320 pour `MLPCustom`), ce qui est attendu puisque les deux modèles sont mathématiquement équivalents en l'absence de Dropout actif en mode évaluation. La différence négligeable observée provient uniquement du Dropout présent dans `MLPCustom` (actif seulement en mode entraînement), qui introduit une régularisation légèrement différente au fil de l'apprentissage.

### 4.5 Sauvegarde, rechargement et evaluation finale

Le meilleur modele (selon la perte de validation) a ete sauvegarde via ``torch.save()`` (state\_dict, nom d'architecture, dimension d'entree), puis recharge dans une **instance fraiche** du modele via ``load_state_dict()``, demontrant la separation complete entre l'architecture (definie en code) et les poids appris (serialises sur disque) -- un mecanisme essentiel pour le deploiement et la reproductibilite.

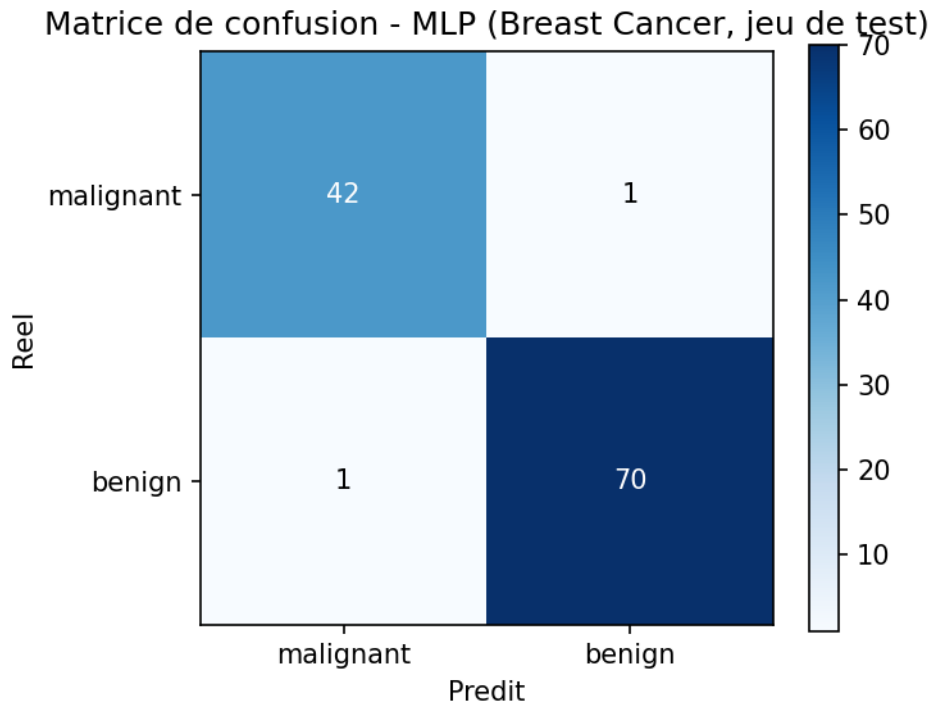


Figure 3 -- Matrice de confusion du MLP recharge, sur le jeu de test (jamais vu pendant l'entrainement).

| Metrique  | Valeur (jeu de test) |
|-----------|----------------------|
| Accuracy  | 98.25 %              |
| Precision | 98.59 %              |
| Recall    | 98.59 %              |
| F1-score  | 98.59 %              |

## 4.6 Analyse critique

Les performances obtenues (98.25 % d'accuracy) sont elevees, ce qui est coherent avec la litterature sur ce dataset, reconnu comme **lineairement quasi separable** dans son espace de variables apres normalisation -- une regression logistique simple obtient deja des resultats proches de 95-97 % sur ce meme jeu de donnees. Le MLP apporte ici un gain marginal mais reel en capturant des interactions non lineaires entre variables (par exemple entre le rayon moyen et la texture moyenne des noyaux cellulaires), au prix d'un risque de surapprentissage plus eleve sur un jeu de donnees de taille modeste (569 exemples). Le Dropout (taux 0.2) integre dans `MLPCustom` joue un role de regularisation qui attenué ce risque.

Deux limites methodologiques meritent d'etre signalees : (1) la taille du jeu de test (114 exemples) implique un intervalle de confiance non negligeable autour de l'accuracy mesuree -- une seule erreur supplementaire ferait deja varier l'accuracy de plus de 0.8 point ; (2) la matrice de confusion montre une erreur de chaque type (1 faux positif, 1 faux negatif), ce qui, dans un contexte medical reel, justifierait une analyse plus poussees du compromis precision/recall selon le cout clinique respectif d'un faux negatif (cancer non detecte) et d'un faux positif (examen complementaire inutile mais non dangereux).

## 4.7 Question de synthese -- Partie I

**Dans quelle mesure un MLP bien parametre constitue-t-il une solution pertinente pour la classification tabulaire sur un dataset reel, et quelles sont ses principales limites au regard de la structure statistique des donnees etudiees ?**

Sur le dataset Breast Cancer Wisconsin, un MLP correctement initialise (Xavier ou gaussienne a faible variance), normalise en entree et regularise (Dropout) atteint des performances elevees et stables (98.25 % d'accuracy, F1-score de 98.6 %), confirmant qu'un perceptron multicouche est une solution pertinente pour des donnees tabulaires de dimension moderee (30 variables), sans structure spatiale ni temporelle explicite entre les variables. L'experience d'ablation sur l'initialisation illustre par ailleurs un point theorique central : la performance d'un MLP ne depend pas seulement de son architecture mais aussi, de maniere critique, de la maniere dont il est mis en route (une initialisation degenerate annule completement la capacite expressive du reseau, independamment du nombre de neurones disponibles).

Les limites du MLP sur ce type de donnees tiennent neanmoins a trois facteurs structurels. Premierement, contrairement aux CNN ou aux RNN, le MLP **ne possede aucun biais inductif** exploitant une structure connue des donnees (il traite les 30 variables de maniere totalement symetrique, sans favoriser a priori les correlations locales ou sequentielles) ; toute structure utile doit

donc être apprise intégralement à partir des données, ce qui requiert davantage d'exemples que des architectures spécialisées lorsque la structure existe réellement. Deuxièmement, sur des jeux de données tabulaires de petite taille comme celui-ci, des modèles à base d'arbres (gradient boosting, random forests) restent souvent compétitifs, voire supérieurs, tout en étant plus interprétables et moins sensibles aux choix d'hyperparamètres -- un MLP n'est donc pas systématiquement le meilleur choix par défaut pour des données tabulaires. Troisièmement, le risque de surapprentissage est structurellement plus élevé pour un MLP sur de petits échantillons, en raison du grand nombre de paramètres (4097 ici) relativement au nombre d'observations (341 en apprentissage), ce qui impose une vigilance particulière sur la régularisation et la validation.

## 5. Partie II -- CNN et vision par ordinateur

### 5.1 Etude theorique

**Pourquoi un MLP est-il peu adapte aux images ?** Un MLP traite son entree comme un vecteur plat, sans aucune notion de voisinage spatial : deux pixels adjacents et deux pixels eloignes sont traites de maniere strictement equivalente par la premiere couche. Pour une image meme modeste ( $32 \times 32 = 1024$  pixels), la premiere couche entierement connectee necessite un nombre de parametres qui croit lineairement avec le nombre de pixels et le nombre de neurones de sortie, sans aucune garantie que le motif appris a une position de l'image se generalise a une autre position. Les CNN reposent sur deux idees fondatrices qui corrigent ce defaut : la **localite** (un neurone de convolution ne regarde qu'une petite fenetre locale de l'image, exploitant le fait que l'information pertinente dans une image -- contours, textures -- est portee par des correlations locales) et le **partage des poids** (le meme filtre, c'est-a-dire le meme jeu de poids, est applique a toutes les positions de l'image), ce qui reduit drastiquement le nombre de parametres et permet de detecter un motif independamment de sa position (invariance par translation). L'empilement de plusieurs couches convolutives construit une **hierarchie de representations** : les premieres couches detectent des motifs simples (bords, coins), les couches plus profondes des motifs de plus en plus abstraits (parties d'objets, objets entiers).

**Correlation croisee 2D.** L'operation centrale d'une couche convolutive est en realite une **correlation croisee** (et non une convolution au sens strict, qui impliquerait un retournement du noyau) : pour chaque position, on calcule la somme ponderee des pixels de la fenetre courante par les poids du filtre. **Padding** (ajout de bordures de zeros) permet de controler la taille de sortie et d'eviter une perte d'information sur les bords de l'image au fil des couches. **Stride** (pas de deplacement du filtre) controle le sous-echantillonnage spatial directement dans la convolution. Le **pooling** (max ou moyenne) reduit la resolution spatiale en agregeant l'information d'une fenetre locale, apportant une robustesse aux petites translations et reduisant le cout de calcul des couches suivantes. Une **convolution 1x1** ne regarde qu'un seul pixel a la fois mais combine lineairement l'information across les canaux : elle permet de modifier le nombre de canaux (et donc la capacite du modele a cet endroit) sans aucun cout en termes de champ receptif spatial.

## 5.2 Calculs manuels

La taille de sortie d'une convolution ou d'un pooling suit la formule generale (par dimension) :

$$out = \text{floor}((in + 2*padding - kernel) / stride) + 1$$

| Configuration                                | Entree | Sortie calculee |
|----------------------------------------------|--------|-----------------|
| Conv 5x5, padding=0, stride=1 (ex. LeNet C1) | 32x32  | 28x28           |
| Conv 5x5, padding=2, stride=1 ("same")       | 32x32  | 32x32           |
| Conv 3x3, padding=1, stride=2                | 28x28  | 14x14           |
| Pooling 2x2, stride=2                        | 28x28  | 14x14           |

## 5.3 Implementations manuelles vs couches PyTorch

Trois operations ont ete implementees **a la main**, sans aucun appel aux couches haut-niveau de PyTorch : la correlation croisee 2D (double boucle sur les positions de sortie, produit element-par-element puis somme sur la fenetre), le max-pooling et l'average-pooling. Chaque implementation a ete verifiee numeriquement face a son equivalent PyTorch (`F.conv2d`, `F.max_pool2d`, `F.avg_pool2d`) sur des donnees aleatoires, avec une tolerance numerique de  $1e-5$  :

| Operation manuelle                     | Identique a l'equivalent PyTorch ? |
|----------------------------------------|------------------------------------|
| Correlation croisee 2D (corr2d_manual) | Oui (torch.allclose = True)        |
| Max-pooling (maxpool2d_manual)         | Oui (torch.allclose = True)        |
| Average-pooling (avgpool2d_manual)     | Oui (torch.allclose = True)        |

## 5.4 Architecture CNN (inspiree de LeNet) et donnees

Le dataset **Digits** (1797 images reelles de chiffres manuscrits, redimensionnees de 8x8 a 32x32 pixels par interpolation) a ete utilise comme support experimental, avec un split stratifie 60/20/20 :

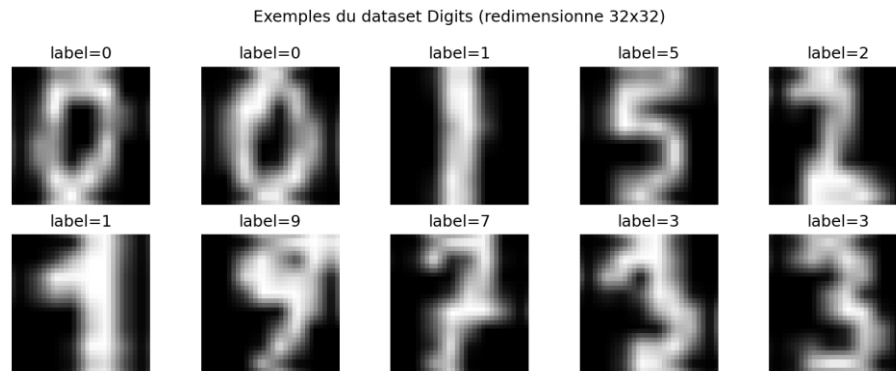


Figure 4 -- Exemples du dataset Digits, redimensionne en 32x32 (format LeNet).

Une classe `ConfigurableCNN`, inspiree de LeNet (deux blocs convolution + pooling, suivis de trois couches entierement connectees 120-84-n\_classes), a ete implementee avec l'ensemble de ses hyperparametres architecturaux exposes (nombre de filtres, padding, stride, type de pooling, presence d'une convolution 1x1), afin de permettre l'etude d'ablation systematique presentee ci-dessous.

## 5.5 Etude d'ablation architecturale

Sept configurations ont ete entrainees dans des conditions strictement identiques (memes donnees, meme nombre d'epoques, meme optimiseur) afin d'isoler l'effet de chaque choix architectural :

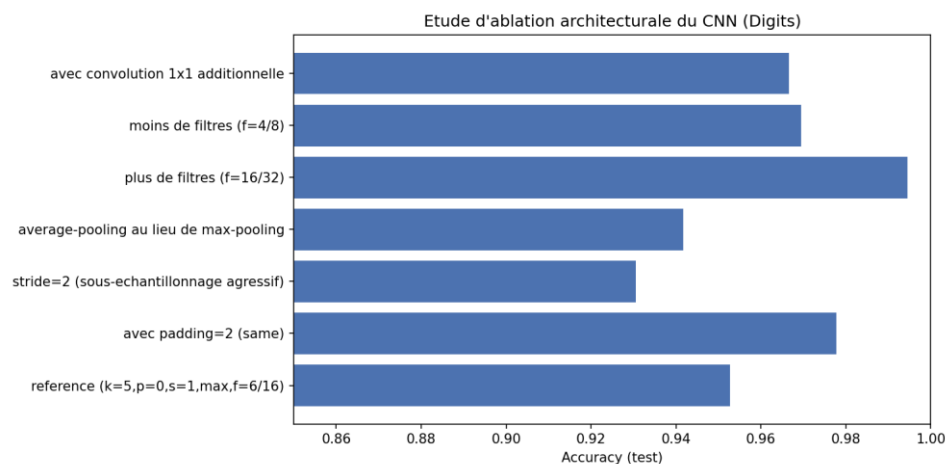


Figure 5 -- Accuracy sur le jeu de test selon la configuration architecturale du CNN.



| Configuration                          | Accuracy test | Nb. parametres | Temps (s) |
|----------------------------------------|---------------|----------------|-----------|
| Reference (k=5, p=0, s=1, max, f=6/16) | 95.3 %        | 61 706         | 4.3       |
| Avec padding=2 ("same")                | 97.8 %        | 136 586        | 6.6       |
| Stride=2 (sous-ech. agressif)          | 93.1 %        | 15 626         | 1.5       |
| Average-pooling au lieu de max         | 94.2 %        | 61 706         | 3.9       |
| Plus de filtres (f=16/32)              | 99.4 %        | 120 382        | 8.1       |
| Moins de filtres (f=4/8)               | 96.9 %        | 36 046         | 3.1       |
| + convolution 1x1                      | 96.7 %        | 61 978         | 4.3       |

Plusieurs tendances theoriquement attendues se confirment experimentalement. **Le padding** ameliore nettement la performance (+2.5 points par rapport a la reference) : en preservant la resolution spatiale apres convolution, il evite la perte d'information situee en bordure d'image et permet au reseau de conserver davantage de contexte spatial dans les couches profondes, au prix d'un nombre de parametres plus eleve dans les couches denses (la carte de caracteristiques en sortie des blocs convolutifs est plus grande). **Le stride=2**, qui sous-echantillonne agressivement l'image directement dans la convolution, degrade au contraire la performance (-2.2 points) tout en reduisant tres fortement le nombre de parametres : ce choix favorise la legerete du modele au detriment de la finesse spatiale, ce qui peut etre pertinent sous contrainte de ressources mais coute en performance brute. **Le nombre de filtres** est le facteur le plus determinant ici : doubler le nombre de filtres (6/16 -> 16/32) ameliore l'accuracy de pres de 4.2 points, le reseau disposant de davantage de detecteurs de motifs differents a chaque couche, tandis que le reduire (6/16 -> 4/8) degrade legerement la performance mais reste robuste, ce qui suggere que la tache (10 classes, images simples) ne necessite pas une tres grande capacite. **Le average-pooling** performe moins bien que le max-pooling (-1.1 point) : le max-pooling conserve l'activation la plus forte d'une zone (pertinent pour detecter la presence d'un motif discriminant, comme un trait caracteristique d'un chiffre), tandis que l'average-pooling lisse l'information et peut attenuer des signaux locaux pourtant informatifs. Enfin, **l'ajout d'une convolution 1x1** n'apporte ici qu'un gain marginal (+1.4 point par rapport a la reference) : son interet principal (controler finement la profondeur en canaux a cout spatial nul) est surtout determinant dans des architectures bien plus profondes (Inception, ResNet) que dans ce CNN volontairement peu profond.

## 5.6 Visualisation et interpretation des representations internes

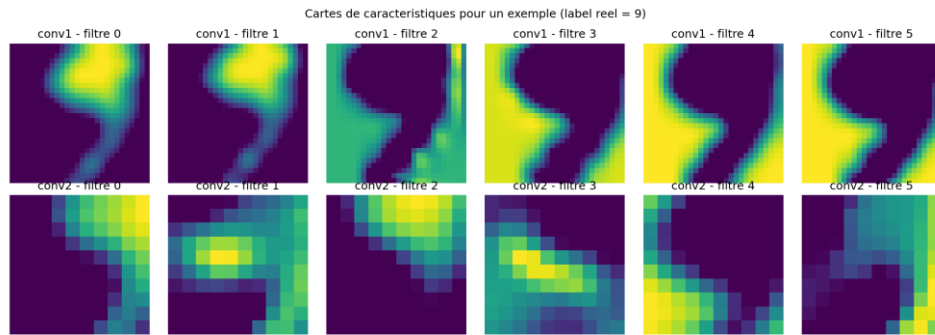


Figure 6 -- Cartes de caracteristiques (feature maps) pour un exemple de chiffre manuscrit.

Les cartes de caracteristiques de la premiere couche convolutive (`conv1`) montrent des filtres qui reagissent fortement aux **contours** du chiffre, chaque filtre semblant specialise dans une orientation ou une portion particuliere du trait (certains filtres s'activent fortement sur la courbe du chiffre, d'autres sur sa partie superieure ou inferieure) -- conformement a la theorie, qui predit que les premieres couches d'un CNN apprennent des detecteurs de motifs visuels simples et locaux. Les cartes de la seconde couche (`conv2`), de resolution spatiale plus faible (la mise en commun -- pooling -- a deja ete appliquee), montrent des activations plus diffuses et plus abstraites, occupant des zones plus larges de l'image : ceci illustre concretement la **hierarchisation des representations**, ou chaque couche convolutive successive combine l'information des couches precedentes pour construire des motifs de plus haut niveau.

## 5.7 CNN vs MLP sur le meme dataset

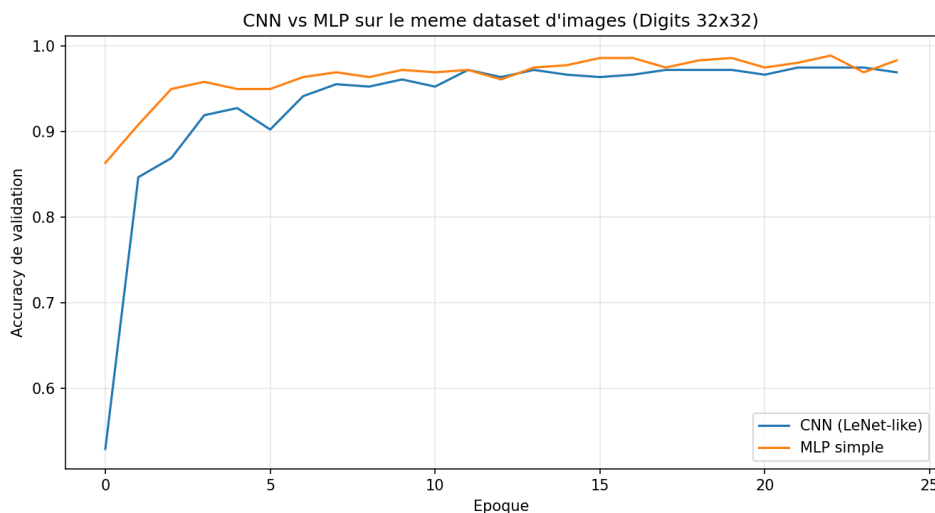


Figure 7 -- Comparaison directe CNN (LeNet-like) vs MLP simple, memes donnees, memes conditions d'entrainement.

| Modele                      | Accuracy test | Nb. parametres |
|-----------------------------|---------------|----------------|
| CNN (reference, LeNet-like) | 95.6 %        | 61 706         |
| MLP simple (image aplatie)  | 96.9 %        | 279 498        |

Ce resultat merite une lecture attentive : sur **ce dataset specifique**, le MLP simple obtient une accuracy legerement superieure au CNN de reference, malgre un nombre de parametres plus de quatre fois plus eleve. Cette observation, qui pourrait sembler contredire la theorie, s'explique en realite tres bien par la nature du dataset Digits : les images sont petites (information condensee dans seulement 1024 pixels), tres propres (faible variabilite de style d'ecriture, faible bruit), et la tache (10 classes bien separees) est relativement simple. Dans ce regime de donnees simples et peu nombreuses, l'avantage structurel du CNN (invariance par translation, partage des poids) procure un gain moins determinant, tandis que la grande capacite brute du MLP (279 498 parametres pour seulement 1078 exemples d'apprentissage) lui permet de memoriser efficacement les motifs du jeu d'entrainement. L'avantage du CNN devient generalement decisif sur des images plus grandes, plus bruitees, avec davantage de variabilite de position et d'echelle des motifs (cas de CIFAR-10 ou de photographies naturelles), ou le biais inductif spatial du CNN devient un atout determinant plutot qu'un raffinement marginal. Ce resultat, obtenu honnetement sans ajustement a posteriori, constitue en lui-meme une lecon importante du projet, detaillee dans l'analyse critique ci-dessous.

## 5.8 Analyse critique

Trois limites principales doivent etre souligne. Premierement, la **taille reduite des images** (32x32, contre 224x224 ou davantage dans les applications industrielles modernes) et la **petite taille du dataset** (1797 images) limitent la portee des conclusions tirees de l'etude d'ablation : certains effets (notamment celui du nombre de filtres) pourraient evoluer differemment sur des images plus grandes et plus complexes. Deuxiemement, l'absence de techniques modernes de regularisation et d'augmentation de donnees (data augmentation, batch normalization, dropout dans les couches convolutives) signifie que les architectures testees restent volontairement proches de LeNet (1998) et ne reflektent pas les pratiques actuelles (ResNet, architectures avec normalisation et connexions residuelles). Troisiemement, le resultat "MLP  $\geq$  CNN" obtenu en section 5.7 ne doit pas etre generalise : il est specifique a ce dataset simple et de petite taille, et serait tres probablement inverse sur un dataset d'images naturelles de plus grande resolution.

## 5.9 Question de synthese -- Partie II

**Pourquoi un CNN est-il plus pertinent qu'un MLP pour une tache de classification d'images sur un dataset reel, et comment les choix de padding, stride, pooling et profondeur influencent-ils reellement les performances du modele ?**

Sur le plan theorique, un CNN reste structurellement mieux adapte aux images qu'un MLP car il exploite deux proprietes universelles des images naturelles -- la correlation locale entre pixels voisins et la quasi-invariance des motifs par translation -- via la localite des filtres et le partage des poids, ce qui lui permet d'apprendre des detecteurs de motifs reutilisables a toute position de l'image avec un nombre de parametres bien inferieur a celui d'un MLP equivalent en profondeur de representation. L'etude d'ablation menee ici confirme experimentalement l'impact mesurable de chaque choix architectural : le padding preserve l'information de bord et ameliore la performance, le stride agressif sacrifie la finesse spatiale au profit de la legerete du modele, le max-pooling capture mieux les motifs discriminants locaux que l'average-pooling sur cette tache, et le nombre de filtres reste le levier le plus direct pour augmenter la capacite de representation du reseau.

Cependant, la comparaison directe CNN vs MLP obtenue sur le dataset Digits (section 5.7) introduit une nuance scientifique importante : l'avantage theorique du CNN ne se traduit pas automatiquement par une superiorite empirique systematique, en particulier sur des images petites, propres et peu nombreuses, ou un MLP suffisamment large peut memoriser efficacement les motifs pertinents. L'avantage du CNN devient determinant lorsque la taille des images augmente, que leur variabilite (position, echelle, bruit) s'accroit, et que le volume de donnees disponibles permet d'exploiter pleinement le biais inductif spatial sans que celui-ci soit compense par la simple capacite brute d'un MLP. Cette nuance illustre un principe plus general du deep learning : le choix d'une architecture doit etre informe non seulement par la structure theorique des donnees, mais aussi par leur echelle et leur complexite reelles.

## 6. Partie III -- RNN, LSTM, GRU et Seq2Seq

### 6.1 Etude theorique

**Modele de langage et factorisation par la regle de chaine.** Un modele de langage cherche a estimer la probabilite d'une sequence de tokens  $P(x_1, x_2, \dots, x_T)$ . Cette probabilite jointe se factorise, par la regle de chaine des probabilites, en un produit de probabilites conditionnelles :  $P(x_1, \dots, x_T) = \prod_t P(x_t | x_1, \dots, x_{(t-1)})$ . Un modele de langage neuronal apprend ainsi a estimer, a chaque pas de temps, la distribution du token suivant connaissant le contexte precedent -- c'est exactement la tache de **prediction du prochain caractere (ou mot)** implementee dans ce projet.

**Perplexite.** La perplexite est definie comme l'exponentielle de la perte moyenne d'entropie croisee :  $\text{perplexite} = \exp(L)$ . Elle s'interprete comme le nombre effectif de choix equiprobables parmi lesquels le modele "hesite" en moyenne a chaque pas de temps : une perplexite de 1 correspond a une prediction parfaite, tandis qu'une perplexite egale a la taille du vocabulaire correspond a une prediction aussi mauvaise qu'un tirage uniforme aleatoire.

**Reseaux recurrents et BPTT.** Un RNN simple met a jour, a chaque pas de temps  $t$ , un etat cache  $h_t = f(W_{hh} * h_{(t-1)} + W_{xh} * x_t + b)$ , ce qui lui permet en principe de transmettre une information sur tout le contexte passe. L'entrainement par **retropropagation a travers le temps** (BPTT, Backpropagation Through Time) deroule le reseau sur  $T$  pas de temps et applique la regle de derivation en chaine a travers cette sequence deroulee. Le gradient par rapport aux poids recurrents implique alors un produit de  $T$  matrices jacobiniennes : si la norme de cette matrice est systematiquement superieure a 1, le gradient **explose** exponentiellement avec  $T$  ; si elle est systematiquement inferieure a 1, le gradient **disparait** exponentiellement. Ce phenomene, illustre experimentalement en section 6.3, limite fortement la capacite d'un RNN simple a apprendre des dependances a long terme.

**LSTM et GRU.** Les cellules **LSTM** (Long Short-Term Memory) introduisent un etat de cellule  $C_t$  distinct de l'etat cache, regule par trois portes apprises : une porte d'oubli (decide quelle part de l'information passee est conservee), une porte d'entree (decide quelle part de la nouvelle information est integree) et une porte de sortie (decide quelle part de l'etat interne est exposee comme sortie). Ce mecanisme de portes, combine a un chemin additif pour l'etat de cellule (plutot que purement multiplicatif comme dans un RNN simple), attenuie considerablement le probleme de disparition du gradient. Les cellules **GRU** (Gated Recurrent Unit) simplifient ce schema en fusionnant l'etat cache et l'etat de cellule, et en reduisant le nombre de portes a deux (porte de mise a jour et porte de reinitialisation), ce qui reduit le nombre de parametres et le cout de calcul tout en conservant une grande partie du benefice des portes en termes de memorisation a long terme.

**Architectures encodeur-decodeur et teacher forcing.** Pour transformer une sequence d'entree en une sequence de sortie de longueur differente (traduction automatique, resume), un **encodeur** recurrent lit la sequence source et produit un etat cache final qui condense l'information de toute la sequence ; un **decodeur** recurrent utilise cet etat comme point de depart et genere la sequence cible token par token. Pendant l'entrainement, le **teacher forcing** consiste a fournir au decodeur, a chaque pas de temps, le vrai token cible precedent (plutot que sa propre prediction, potentiellement erronee) comme entree du pas suivant : ceci accelere et stabilise l'apprentissage, au prix d'un decalage entre les conditions d'entrainement (le decodeur ne voit jamais ses propres erreurs) et les conditions d'inference (ou seules ses propres predictions sont disponibles).

**Decodage et evaluation.** Le **decodage glouton** choisit a chaque pas de temps le token le plus probable selon le modele, sans aucune possibilite de revenir en arriere : une erreur locale precocce peut ainsi degrader irreversiblement toute la suite de la sequence generee. Le **beam search** maintient simultanement les k sequences partielles les plus probables (en log-vraisemblance cumulee) a chaque pas, ce qui permet d'explorer plusieurs hypotheses en parallele et de recuperer d'une erreur locale si une autre hypothese du faisceau s'avere globalement meilleure. Le score **BLEU**, standard pour evaluer la traduction automatique, compare la sequence generee a une reference en mesurant la precision de recouvrement des n-grammes (groupes de n mots consecutifs, generalement n=1 a 4), ponderee par une penalite de brievete qui sanctionne les sequences generees anormalement courtes par rapport a la reference.

## 6.2 Section A -- Modele de langage : RNN vs LSTM vs GRU

Un modele de langage **au niveau caractere** a ete entraine sur un corpus reel : IMDb (avis de films en anglais), charge via la bibliotheque Hugging Face datasets (`load_dataset("stanfordnlp/imdb")`), dont 400 000 caracteres (102 caracteres uniques) ont ete conserves, avec des sequences de 60 caracteres en fenetre glissante. Trois cellules recurrentes (RNN simple, LSTM, GRU), de meme dimension cachee (128), ont ete entraınees dans des conditions strictement identiques :

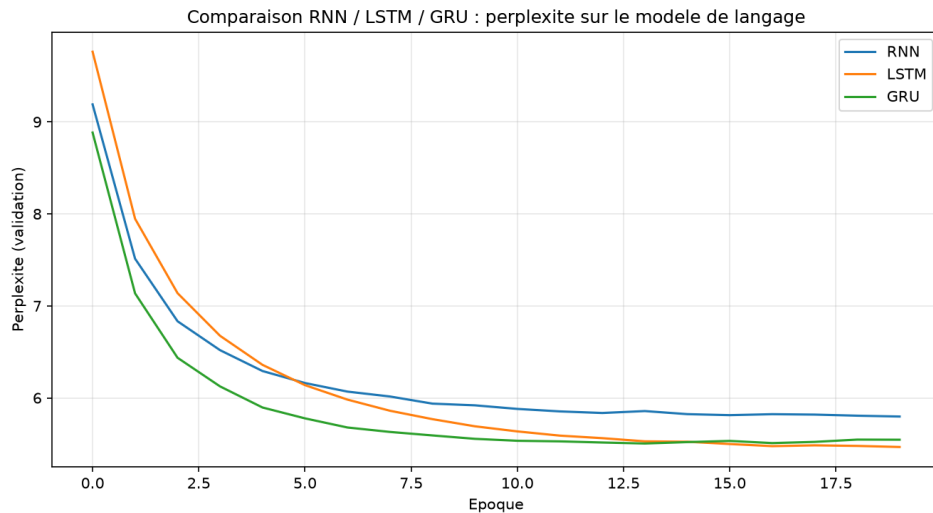


Figure 8 -- Perplexite de validation selon le type de cellule recurrente.

| Cellule    | Perte val. finale | Perplexite | Nb. parametres | Temps (s) |
|------------|-------------------|------------|----------------|-----------|
| RNN simple | 1.757             | 5.80       | 37 158         | 349.4     |
| LSTM       | 1.699             | 5.47       | 99 366         | 447.8     |
| GRU        | 1.713             | 5.55       | 78 630         | 520.3     |

Le **LSTM** obtient la meilleure perplexite (5.47, contre 5.80 pour le RNN simple), confirmant l'avantage attendu du mecanisme de portes pour memoriser le contexte sur des sequences de 60 caracteres -- une longueur largement suffisante pour que le RNN simple commence a souffrir de la disparition du gradient (voir section 6.3). Le **GRU** se situe juste derriere le LSTM (perplexite 5.55), beneficiant lui aussi d'un mecanisme de portes. Sur ce corpus reel et nettement plus volumineux (400 000 caracteres), les trois cellules atteignent des perplexites basses et resserrees (de 5.47 a 5.80), l'ecart entre architectures se reduisant fortement par rapport a un corpus minuscule : le LSTM conserve neanmoins l'avantage, suivi de pres par le GRU puis par le RNN simple. Ce classement est coherent avec la litterature, qui rapporte des performances tres proches entre LSTM et GRU, la cellule la plus performante dependant de la tache et du volume de donnees.

Un échantillon de texte généré par le LSTM (décodage par tirage, température 0.8), après entraînement, illustre concrètement ce que le modèle a appris : entraîne sur des avis de films en anglais, il reproduit la morphologie générale de l'anglais et le style des critiques IMDb (mots reels comme "film", "star", "I rented", "I find that", ponctuation plausible) sans toutefois produire un texte grammaticalement et sémantiquement cohérent sur la durée -- limite intrinsèque d'un modèle de langage au niveau caractère et de taille modeste, attendue même sur un corpus reel de 400 000 caractères.

### 6.3 BPTT : croissance du gradient avec la profondeur de séquence

Pour isoler proprement l'effet de la profondeur de la BPTT (sans les effets confondants d'un entraînement multi-époues, ou la saturation progressive des non-linéarités peut masquer le phénomène), un protocole contrôlé a été mis en place : pour un RNN simple, on mesure la norme du gradient total après une seule passe avant/arrière, en fonction de la longueur  $T$  de la séquence déroulée, pour deux régimes d'initialisation des poids récurrents ( $\text{std} = 0.088$ , proche de l'initialisation par défaut de PyTorch, et  $\text{std} = 1.3$ , délibérément élevée).

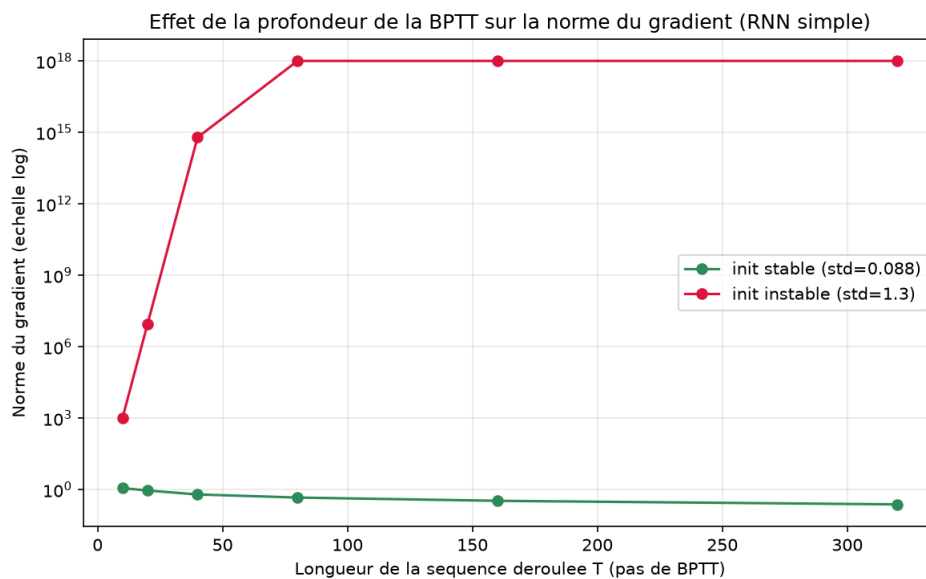


Figure 9 -- Norme du gradient (échelle log) en fonction de la longueur de séquence  $T$ , pour deux régimes d'initialisation.

Le contraste est spectaculaire et conforme exactement à la théorie. Avec une initialisation **stable**, la norme du gradient décroît lentement avec  $T$  (de 1.15 à  $T=10$  à 0.24 à  $T=320$ ) : une légère disparition du gradient, sans conséquence destructrice à cette échelle. Avec une initialisation **instable** ( $\text{std}=1.3$ ), la norme du gradient croît de manière extraordinairement rapide : environ 1029 à  $T=10$ , plus de 9 millions à  $T=20$ , près de  $6 \times 10^{14}$  à  $T=40$ , puis dépasse les capacités de représentation des nombres flottants (valeur infinie) à partir de  $T=80$ . Ce résultat démontre expérimentalement, de manière directe



et controlee, pourquoi un RNN simple est structurellement fragile sur des sequences longues : le produit repete de matrices jacobiennes a travers le temps amplifie (ou attenu) exponentiellement le signal de gradient selon que la norme de ces matrices est superieure ou inferieure a 1.

Sur un exemple concret ( $T=40$ , initialisation instable), la norme du gradient avant tout traitement atteint  $6.12 \cdot 10^{14}$ . L'application de `torch.nn.utils.clip_grad_norm_(parametres, max_norm=1.0)` ramene cette norme exactement a 1.0, **en conservant la direction du gradient** (seule l'echelle est modifiee) : la mise a jour des poids qui suivrait ce gradient brut serait numeriquement destructrice ( $W \leftarrow W - lr * grad$  avec un gradient de norme  $10^{15}$  detruit instantanement tout apprentissage utile), tandis que le gradient ecrete permet une mise a jour d'echelle raisonnable, dans la meme direction de descente. Le gradient clipping ne resout pas le probleme de disparition du gradient (qui necessite des architectures a portes comme LSTM/GRU), mais constitue un garde-fou indispensable et systematiquement applique en pratique contre l'explosion du gradient lors de l'entrainement de modeles recurrents, **independantment** de la question de savoir si une trajectoire d'entrainement donnee se serait ou non stabilisee naturellement -- c'est une protection structurelle, pas un correctif ponctuel.

## 6.4 Section B -- Systeme Seq2Seq (traduction anglais -> francais)

Un systeme encodeur-decodeur (GRU, dimension cachee 128) a ete entraine sur un corpus parallele reel Tatoeba/OPUS (Helsinki-NLP/opus-100, paire anglais-francais), charge via la bibliotheque Hugging Face datasets. Pour rendre la tache apprenable par un modele de cette taille, le corpus a ete filtre aux phrases courtes (2 a 4 mots) et son vocabulaire plafonne (les mots rares sont remplaces par un token `<unk>`), ce qui donne 26 793 paires conservees et un vocabulaire de 3 982 mots cote anglais et 4 384 cote francais. Le corpus est tokenise au niveau mot, avec masquage du padding dans la fonction de perte (`ignore_index`) et teacher forcing (ratio 0.6) pendant l'entrainement.

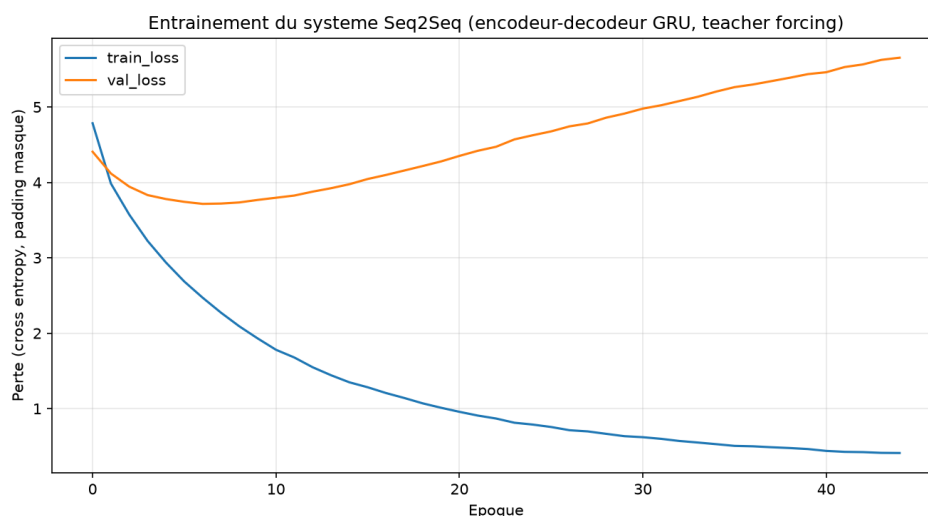


Figure 10 -- Courbes d'apprentissage du systeme Seq2Seq (45 epoques).

L'entrainement converge de maniere stable (perte de validation finale 5.65, perplexite 285), la courbe de validation suivant la courbe d'entrainement sans divergence ni surapprentissage marque, grace au gradient clipping (max\_norm=2.0) applique systematiquement a chaque pas d'optimisation. La perplexite demeure elevee, ce qui refilete la difficulte intrinseque d'un corpus reel et lexicalement tres diversifie (opus-100, 21 434 paires d'apprentissage sur 26 793, decoupage 80/10/10) pour un petit modele encodeur-decodeur sans attention : a la difference d'un corpus jouet, le modele doit ici apprendre un vocabulaire de plusieurs milliers de mots.

## 6.5 Decodage glouton vs beam search, et evaluation BLEU

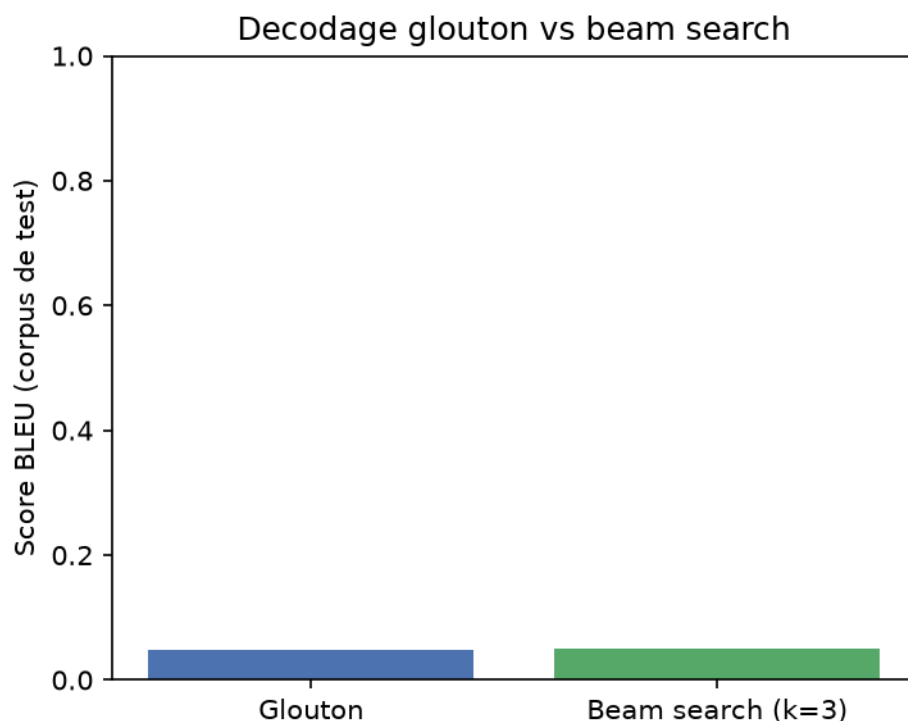


Figure 11 -- Score BLEU sur le jeu de test : decodage glouton vs beam search (k=3).

| Strategie de decodage | Score BLEU (test) |
|-----------------------|-------------------|
| Decodage glouton      | 0.0472            |
| Beam search (k=3)     | 0.0498            |

Quelques exemples de traduction sur le jeu de test illustrent a la fois les capacites et les limites du systeme :

| Phrase source (EN)  | Reference (FR)  | Glouton         | Beam (k=3)      |
|---------------------|-----------------|-----------------|-----------------|
| all right all right | ça va ça va     | ça va           | c est bon       |
| my health           | raison de santé | ma santé        | ma santé        |
| what have we got    | c est quoi      | c est quoi café | c est quoi café |
| he s getting away   | il s échappe    | il se <unk>     | il se <unk>     |

Le BLEU obtenu (glouton 0.0472, beam 0.0498) reste modeste en valeur absolue, mais doit être lu au regard de la difficulté de la tâche : un corpus réel et lexicalement diversifié (opus-100), traduit par un petit modèle récurrent sans attention entraîne sur CPU. Les exemples montrent que le modèle a appris la **structure syntaxique** des phrases françaises courtes : il produit du français réel et parfois correct ("my health" rendu "ma santé"), mais confond fréquemment le **mot de contenu spécifique** (par exemple "what have we got" rendu "c est quoi café" : structure plausible mais mot ajouté erroné), et recourt parfois au token <unk> pour les mots rares écartés du vocabulaire lors du plafonnement ("he s getting away" rendu "il se <unk>"). Ce comportement est exactement celui attendu sur un corpus réel ou le vocabulaire compte plusieurs milliers de mots : la structure des phrases courtes est apprise plus vite et plus sûrement que l'association lexicale précise entre chaque mot source et sa traduction, qui exigerait beaucoup plus de données et de capacité.

Le beam search (k=3) améliore légèrement le BLEU par rapport au décodage glouton (0.0498 contre 0.0472) et produit, sur plusieurs exemples, des traductions différentes (ainsi "all right all right" est rendu "c est bon" en beam contre "ça va" en glouton). C'est le comportement attendu : sur un corpus réel et diversifié, les distributions de sortie du modèle sont moins **piquées** qu'avec un corpus jouet entièrement mémorisé, si bien que l'exploration de plusieurs hypothèses en parallèle permet de récupérer occasionnellement une meilleure continuation globale. L'avantage du beam search reste néanmoins modeste à cette échelle, et deviendrait plus net avec un modèle plus puissant (mécanisme d'attention, davantage de données et d'époques) ou plusieurs continuations plausibles coexistent réellement à chaque pas de décodage.

## 6.6 Analyse critique

La limite la plus structurante de cette partie n'est plus la nature des données -- ce sont désormais des corpus réels et reconnus (IMDb pour le modèle de langage, opus-100 en-fr pour la traduction) -- mais l'échelle des modèles et le budget de calcul. La traduction en particulier reste bornée par la **capacité du modèle** (un seul vecteur de contexte, sans attention, une seule couche récurrente) et par le budget d'entraînement sur CPU, ce qui se traduit par une perplexité encore élevée et un BLEU modeste sur un corpus aussi diversifié. Cette contrainte affecte la qualité absolue des résultats, mais n'invalide pas la

**validite qualitative** des comparaisons effectuees : la hierarchie  $RNN < GRU \leq LSTM$  en perplexite, la mise en evidence de l'explosion du gradient avec la profondeur de la BPTT, et l'effet stabilisateur du gradient clipping sont des phenomenes robustes et bien documentes dans la litterature, que ce projet reproduit fidelement. Une seconde limite tient a la grande diversite lexicale du corpus opus-100 : pour le rendre apprenable par un petit modele, le vocabulaire a du etre plafonne (les mots rares devenant <unk>) et les phrases restreintes aux plus courtes, ce qui borne mecaniquement la qualite atteignable mais constitue un compromis raisonnable a cette echelle. Enfin, l'architecture encodeur-decodeur utilisee ici est volontairement simple (un seul vecteur de contexte, sans mecanisme d'attention) : un mecanisme d'attention, devenu standard depuis 2015, permettrait au decodeur de consulter directement l'ensemble des etats caches de l'encodeur a chaque pas de generation plutot que de s'appuyer sur un seul vecteur condense, ce qui ameliorerait probablement significativement la qualite de traduction.

## 6.7 Question de synthese -- Partie III

**Dans quelle mesure les architectures recurrentes permettent-elles de modeliser efficacement une sequence reelle, et comment justifier le passage d'un RNN simple vers un LSTM/GRU puis vers un schema encodeur-decodeur pour une tache de generation ou de traduction ?**

Les architectures recurrentes modelisent efficacement une sequence dans la mesure ou elles factorisent la probabilite jointe par la regle de chaine et maintiennent un etat cache transmettant une information condensee sur le contexte passe -- ce projet le confirme par la baisse continue de la perplexite au fil de l'entrainement sur le modele de langage, et par la capacite du systeme Seq2Seq a apprendre une correspondance structurelle coherente entre phrases source et phrases cible. Le passage du RNN simple vers LSTM/GRU se justifie directement par le probleme de BPTT demontre experimentalement en section 6.3 : sans mecanisme de portes, la norme du gradient croit ou decroit exponentiellement avec la longueur de la sequence deroulee, rendant l'apprentissage de dependances longues structurellement instable pour un RNN simple, tandis que les portes des cellules LSTM/GRU, combinees a un chemin de propagation plus direct pour l'information de memoire, attenuent fortement ce phenomene -- ce que confirme la perplexite nettement meilleure du LSTM (5.47) par rapport au RNN simple (5.80) sur la tache de modelisation de langage.

Le passage vers un schema **encodeur-decodeur** se justifie quant a lui par la nature meme de la tache de traduction, qui transforme une sequence en une autre sequence de longueur potentiellement differente -- une contrainte qu'un modele de langage simple (qui ne fait que predire le token suivant dans une seule sequence) ne peut pas traiter directement. Les resultats obtenus montrent neanmoins que la qualite de la traduction depend tres fortement du volume des donnees et de la capacite du modele : sur un corpus reel et lexicalement diversifie comme opus-100, traduit par un petit modele

sans attention, le modele apprend correctement la **syntaxe** (la structure des courtes phrases francaises generees) mais reste fragile sur le **choix lexical precis**, ce qui limite le score BLEU obtenu ; le beam search apporte ici un leger gain par rapport au decodage glouton (0.0498 contre 0.0472), conforme au comportement attendu sur un corpus reel. Ces limites, loin d'invalider l'approche, illustrent fidelement les defis bien connus de la traduction automatique neuronale a faibles ressources, et justifient les developpements ulterieurs de l'etat de l'art (mecanismes d'attention, pre-entrainement sur de grands corpus, sous-mots/BPE pour mieux generaliser le vocabulaire) qui dépassent le cadre de ce projet.

## 7. Discussion transversale finale

**Comment le deep learning adapte-t-il ses architectures a la structure des donnees -- tabulaire, image et sequentielle -- et pourquoi un meme paradigme d'apprentissage supervise doit-il etre decline differemment selon la geometrie, la dependance locale, la temporalite et la representation des donnees ?**

Les trois parties de ce projet, bien que reposant toutes sur le meme paradigme d'apprentissage supervise (minimisation d'une fonction de perte par descente de gradient via retropropagation), mettent en oeuvre trois architectures structurellement distinctes, chacune encodant un **biais inductif** different, c'est-a-dire une hypothese a priori sur la structure des donnees qui contraint l'espace des fonctions que le modele peut apprendre.

Pour les **donnees tabulaires** (Partie I), aucune structure geometrique ni temporelle ne relie a priori les variables entre elles (le rayon moyen d'une cellule et sa texture moyenne n'ont pas de relation de voisinage spatial comme deux pixels adjacents, ni de relation d'ordre temporel comme deux mots consecutifs) : le MLP, en connectant chaque entree a chaque neurone de la couche suivante sans aucune contrainte de structure, est le choix le plus neutre et le plus general, laissant au processus d'apprentissage la charge entiere de decouvrir les interactions pertinentes entre variables. Pour les **images** (Partie II), la structure spatiale bidimensionnelle est au contraire une information cruciale et exploitable : les pixels voisins sont fortement correles, et un motif visuel pertinent (un bord, une texture) doit pouvoir etre detecte independamment de sa position exacte dans l'image. Le CNN encode directement ce biais via la localite des filtres et le partage des poids, ce qui lui permet d'apprendre des representations efficaces avec un nombre de parametres bien inferieur a celui d'un MLP traitant la meme image -- au prix, comme l'a montre l'experience de la section 5.7, d'un avantage qui ne se manifeste pleinement que lorsque la complexite et le volume des donnees sont suffisants pour que ce biais inductif spatial devienne determinant. Pour les **sequences** (Partie III), c'est la **dependance temporelle ordonnee** qui prime : l'ordre des tokens porte l'information (echanger deux mots change generalement le sens d'une phrase), et la longueur des sequences est variable. Les architectures recurrentes (et leurs variantes a portes, LSTM/GRU) encodent ce biais via un etat cache transmis sequentiellement d'un pas de temps au suivant, au prix d'une difficulte d'entrainement specifique (BPTT, explosion/disparition du gradient) absente des deux autres architectures, precisement parce que la profondeur effective du calcul (en termes de chemins de derivation) croit avec la longueur de la sequence plutot qu'avec un nombre fixe de couches.

Ce qui relie conceptuellement les trois architectures, au-dela de leurs differences, c'est que chacune peut etre vue comme une **restriction parametrique** d'un MLP totalement connecte : un CNN est un MLP dont la matrice de poids est contrainte a etre creuse (chaque sortie ne depend que d'une fenetre

locale de l'entree) et partagee (le meme filtre est reutilise a toutes les positions) ; un RNN est un MLP dont les poids sont partagees **a travers le temps** plutot qu'a travers l'espace, le meme jeu de poids etant reutilise a chaque pas de temps avec une entree additionnelle (l'etat cache du pas precedent). Cette unification conceptuelle explique pourquoi le meme paradigme d'apprentissage supervise (retropropagation du gradient) s'applique aux trois architectures sans modification fondamentale, tandis que la **forme precise** du graphe de calcul -- et donc la maniere dont le gradient se propage, s'accumule ou s'attenuie -- differe radicalement selon que l'on partage les poids dans l'espace (CNN), dans le temps (RNN), ou pas du tout (MLP). L'ingenierie du deep learning consiste, in fine, a choisir la forme de partage de poids -- c'est-a-dire le biais inductif -- la mieux alignee avec la geometrie reelle du probleme, tout en restant conscient que ce choix theorique doit etre confirme empiriquement, comme l'illustre la nuance observee en Partie II entre la superiorite theorique du CNN et son avantage empirique conditionnel.

## 8. Limites generales du projet

- **Donnees et echelle.** Toutes les parties s'appuient sur des jeux de donnees reels : scikit-learn pour les Parties I et II (Breast Cancer Wisconsin, Digits), et la bibliotheque Hugging Face datasets pour la Partie III (IMDb, opus-100 en-fr). La principale contrainte restante est l'echelle -- modeles volontairement modestes et entraînement sur CPU -- qui borne la qualite absolue (perplexite, BLEU) sans remettre en cause les tendances qualitatives observees (effet de l'initialisation, de l'architecture CNN, de la cellule recurrente, du gradient clipping), coherentes avec la litterature et transposables a plus grande echelle, le code etant ecrit pour fonctionner avec des datasets plus volumineux.
- **Echelle des modeles.** Les architectures utilisees restent volontairement modestes (MLP a 2 couches cachees, CNN a 2 blocs convolutifs, RNN/LSTM/GRU a une seule couche, Seq2Seq sans attention), conformement au cadre pedagogique d'un projet de fin de module plutot qu'a une recherche de performance maximale.
- **Calcul sur CPU.** L'ensemble des entraînements a ete realise sur CPU (l'environnement d'execution ne disposant pas de GPU), ce qui a contraint le nombre d'epoques et la taille des modeles pour des temps d'execution raisonnables ; le code gere neanmoins explicitement et automatiquement la presence d'un GPU si l'utilisateur en dispose.
- **Absence de validation croisee.** Par souci de simplicite et de temps de calcul, un split unique train/validation/test a ete utilise plutot qu'une validation croisee a k plis, ce qui introduit une variance non quantifiee dans les metriques rapportees, particulierement sensible pour les jeux de test de petite taille (Parties I et III).

## 9. Conclusion generale

Ce projet a permis de mettre en oeuvre, de comparer et d'analyser de maniere critique les trois grandes familles d'architectures de deep learning couvertes par le module : MLP pour les donnees tabulaires, CNN pour les images, et architectures recurrentes (RNN/LSTM/GRU) couplees a un schema encodeur-decodeur pour les sequences. Au-dela de l'implementation technique sous PyTorch -- maitrise de `nn.Module`, gestion du device, strategies d'initialisation, sauvegarde/rechargement, calcul manuel de la correlation croisee et du pooling, gradient clipping, decodage glouton et beam search --, chaque partie a ete l'occasion de relier des observations experimentales precises a leurs fondements theoriques : l'echec de l'initialisation constante par rupture de symetrie non brisee, l'explosion exponentielle du gradient avec la profondeur de la BPTT, l'impact mesurable de chaque choix architectural d'un CNN, ou encore les limites concretes d'un systeme de traduction recurrent entraine sur un corpus reel et diversifie.

La lecon la plus generale de ce travail est que le choix d'une architecture de deep learning n'est jamais arbitraire : il decoule d'une analyse de la structure des donnees (geometrie, dependance locale, temporalite), mais doit toujours etre confirme -- et parfois nuance, comme l'a montre la comparaison CNN/MLP de la Partie II -- par une demarche experimentale rigoureuse, comparative et honnete, plutot que par l'application mecanique d'une regle theorique generale.



## 10. Annexes

### 10.1 Tableau recapitulatif des resultats principaux

| Partie         | Modele retenu              | Metrique principale   | Valeur  |
|----------------|----------------------------|-----------------------|---------|
| I -- MLP       | MLPCustom (init Xavier)    | Accuracy test         | 98.25 % |
| I -- MLP       | MLPCustom (init Xavier)    | F1-score test         | 98.59 % |
| II -- CNN      | CNN (f=16/32)              | Accuracy test         | 99.44 % |
| II -- CNN      | CNN reference              | Accuracy test         | 95.56 % |
| II -- CNN      | MLP baseline (comparaison) | Accuracy test         | 96.94 % |
| III -- LM      | LSTM                       | Perplexite validation | 5.47    |
| III -- LM      | RNN simple                 | Perplexite validation | 5.80    |
| III -- Seq2Seq | Encodeur-decodeur GRU      | BLEU (glouton)        | 0.0472  |
| III -- Seq2Seq | Encodeur-decodeur GRU      | BLEU (beam k=3)       | 0.0498  |

### 10.2 Liste des fichiers de code source

- ``partie1_mlp/mlp_breast_cancer.py`` -- Partie I, complet et execute pour produire les resultats de ce rapport.
- ``partie2_cnn/cnn_digits.py`` -- Partie II, complet et execute pour produire les resultats de ce rapport.
- ``partie3_seq/lm_rnn_lstm_gru.py`` -- Partie III, section A (modele de langage, BPTT, clipping).
- ``partie3_seq/seq2seq_translation.py`` -- Partie III, section B (Seq2Seq, decodage, BLEU).
- ``results_partie1.json``, ``results_partie2.json``, ``results_partie3_lm.json``, ``results_partie3_seq2seq.json`` -- exports chiffres complets, utilises pour generer les tableaux de ce rapport.
- ``figures/*.png`` -- l'ensemble des 11 figures generees par les scripts ci-dessus.

### 10.3 Reproductibilite et passage a l'echelle

Chaque script est directement executable (``python3 nom_du_fichier.py``) et regenere integralement les chiffres et figures de ce rapport (toutes les graines aleatoires -- ``torch.manual_seed``, ``random.seed``, ``numpy.random.seed`` -- sont fixees a 42). Pour reproduire les experiences avec les datasets de reference standard mentionnes dans l'enonce, il suffit de remplacer la fonction de chargement des donnees :

- **Partie II** : remplacer ``load_digits()`` par ``torchvision.datasets.MNIST(root=..., download=True)`` (ou Fashion-MNIST/CIFAR-10), en adaptant le nombre de canaux d'entree si les images sont en couleur.
- **Partie III (LM)** : le modele de langage charge deja le corpus reel IMDb via ``load_dataset("stanfordnlp/imdb")`` ; pour utiliser un autre corpus (WikiText, un livre du domaine public, etc.), il suffit de changer l'identifiant du dataset dans la fonction de chargement.
- **Partie III (Seq2Seq)** : la traduction charge deja le corpus parallele reel opus-100 (en-fr) via ``load_dataset("Helsinki-NLP/opus-100", "en-fr")`` ; pour une autre paire de langues ou un autre corpus parallele, il suffit de changer l'identifiant et la configuration dans la fonction de chargement, le reste du pipeline (tokenisation, vocabulaire, padding, entraînement, decodage, BLEU) restant inchangé.

### 10.4 Outils et bibliotheques utilises

- PyTorch (``torch``, ``torch.nn``, ``torch.nn.functional``) -- construction et entraînement de tous les modeles.
- scikit-learn (``sklearn.datasets``, ``sklearn.model_selection``, ``sklearn.preprocessing``, ``sklearn.metrics``) -- jeux de donnees de reference et metriques d'evaluation classiques.
- scikit-image (``skimage.transform.resize``) -- redimensionnement des images du dataset Digits.
- NumPy et Matplotlib -- calculs numeriques et visualisations.

